Relational Database Design and Anomalies

FAST PAST PAPERS SOLVED + NOTES

1 Inventory Management Normalization Example

1.1 Given Table (Unnormalized)

ProductID	ProductName	UnitPrice	QuantityInStock	CategoryID	CategoryName	SupplierID
P123	Nike56	4300	100	C1	Shoes	S1
P345	Skechers	12000	50	C1	Shoes	S2
P678	Nike57	5600	100	C2	Bags	S1
P900	Puma87	12000	190	C1	Shoes	S3
P680	Puma89	4500	80	C2	Bags	S3
P569	Addidas4	8000	78	C3	Caps	S4
P340	Reebok8	10000	1000	C4	Gym Kits	S5
P700	Addidas1	6700	800	C1	Shoes	S4

Table 1: Unnormalized Product Table

1.2 Step 1: Insert, Update, and Delete Anomalies

- **Insert Anomaly:** Cannot add a new product without specifying category and supplier information.
- **Update Anomaly:** Changing supplier email or phone requires updating multiple rows.
- **Delete Anomaly:** Deleting the last product of a supplier or category may remove the supplier or category information.

1.3 Step 2: Primary Key

- **Primary Key (PK):** ProductID

1.4 Step 3: Functional Dependencies

- Product-level FD: $ProductID \rightarrow ProductName, UnitPrice, QuantityInStock, CategoryID, SupplierID$

- Category-level FD: $CategoryID \rightarrow CategoryName$
- Supplier-level FD: $SupplierID \rightarrow SupplierName, Email, Phone$

1.5 Step 4: 3NF Decomposition

Step 4.1: 1NF - Already satisfied (all attributes atomic).

Step 4.2: 2NF - Already satisfied (PK = ProductID, single attribute, so no partial dependency).

Step 4.3: 3NF - Remove transitive dependencies (CategoryName depends on CategoryID, Supplier details depend on SupplierID).

3NF Relations:

- **Product(ProductID, ProductName, UnitPrice, QuantityInStock, CategoryID, SupplierID)**
- **Category(CategoryID, CategoryName)**
- **Supplier(SupplierID, SupplierName, Email, Phone)**

1.6 Step 5: Relational Schema and Referential Integrity

- **Product Table:**
  - PK: ProductID
  - FK: CategoryID references Category(CategoryID)
  - FK: SupplierID references Supplier(SupplierID)
- **Category Table:** PK = CategoryID
- **Supplier Table:** PK = SupplierID

1.7 Step 6: SQL Representation

```
CREATE TABLE Category (
    CategoryID CHAR(3) PRIMARY KEY,
    CategoryName VARCHAR(50) NOT NULL
);

CREATE TABLE Supplier (
    SupplierID CHAR(3) PRIMARY KEY,
    SupplierName VARCHAR(50) NOT NULL,
    Email VARCHAR(50),
    Phone VARCHAR(20)
);
```

```
CREATE TABLE Product (
    ProductID CHAR(5) PRIMARY KEY,
    ProductName VARCHAR(50) NOT NULL,
    UnitPrice DECIMAL(10,2),
    QuantityInStock INT,
    CategoryID CHAR(3),
    SupplierID CHAR(3),
    FOREIGN KEY (CategoryID) REFERENCES Category(CategoryID),
    FOREIGN KEY (SupplierID) REFERENCES Supplier(SupplierID)
);
```

1.8 Observations

- All insert, update, delete anomalies are removed.
- Each table has a clear primary key.
- Referential integrity ensures valid category and supplier links.

2 Inventory Management Normalization Example (Multiple Suppliers per Product)

2.1 Scenario and Constraint

We consider the case where a single product can be supplied by multiple suppliers. **Example:** Nike56 (Category C1) is supplied by both S1 (Nike) and S2 (Skechers).

Implication: - The original ProductID alone cannot uniquely identify a row in the inventory table, because the same product may appear multiple times with different suppliers. - To uniquely identify each row, we need a **composite primary key: (ProductID, SupplierID)**.

Reason for composite PK: - ProductID identifies the product. - SupplierID identifies which supplier is supplying that product. - Together, they ensure uniqueness for each product-supplier combination.

2.2 Unnormalized Table

ProductID	ProductName	UnitPrice	QuantityInStock	CategoryID	CategoryName	SupplierID
P123	Nike56	4300	100	C1	Shoes	S1
P123	Nike56	4200	80	C1	Shoes	S2
P678	Nike57	5600	100	C2	Bags	S1
P900	Puma87	12000	190	C1	Shoes	S3

Table 2: Unnormalized Inventory Table (Multiple Suppliers per Product)

2.3 Step 1: Insert, Update, and Delete Anomalies

- **Insert Anomaly:** Cannot add a new product-supplier combination without knowing both supplier and category details.
- **Update Anomaly:** Changing supplier contact info requires updating multiple rows for all products supplied by that supplier.
- **Delete Anomaly:** Deleting a product-supplier combination could remove supplier or product info if it is the last occurrence.

2.4 Step 2: Primary Key

- Composite Primary Key: $(ProductID, SupplierID)$ - Justification: Together, these two attributes uniquely identify a product-supplier row.

2.5 Step 3: Functional Dependencies

- Product-level FD: $ProductID \rightarrow ProductName, CategoryID$
- Category-level FD: $CategoryID \rightarrow CategoryName$
- Supplier-level FD: $SupplierID \rightarrow SupplierName, Email, Phone$
- Inventory FD: $(ProductID, SupplierID) \rightarrow UnitPrice, QuantityInStock$

2.6 Step 4: 3NF Decomposition

Step 4.1: 1NF - All attributes are atomic, so 1NF is satisfied.

Step 4.2: 2NF - Check for partial dependency on part of the composite key $(ProductID, SupplierID)$: - ProductName, CategoryID depend only on ProductID $\rightarrow$ partial dependency exists. - SupplierName, Email, Phone depend only on SupplierID $\rightarrow$ partial dependency exists. - Remedy: Decompose tables.

Step 4.3: 3NF - Remove transitive dependencies: CategoryName depends on CategoryID. - Supplier details depend on SupplierID.

Resulting 3NF Relations:

- **Product(ProductID, ProductName, CategoryID)**
- **Category(CategoryID, CategoryName)**
- **Supplier(SupplierID, SupplierName, Email, Phone)**
- **ProductSupplier(ProductID, SupplierID, UnitPrice, QuantityInStock)**
  - **Primary Key:** (ProductID, SupplierID)
  - **Foreign Keys:** - ProductID references Product(ProductID) - SupplierID references Supplier(SupplierID)

2.7 Step 5: SQL Representation

```
CREATE TABLE Category (
    CategoryID CHAR(3) PRIMARY KEY,
    CategoryName VARCHAR(50) NOT NULL
);

CREATE TABLE Supplier (
    SupplierID CHAR(3) PRIMARY KEY,
    SupplierName VARCHAR(50) NOT NULL,
    Email VARCHAR(50),
    Phone VARCHAR(20)
);

CREATE TABLE Product (
    ProductID CHAR(5) PRIMARY KEY,
    ProductName VARCHAR(50) NOT NULL,
    CategoryID CHAR(3),
    FOREIGN KEY (CategoryID) REFERENCES Category(CategoryID)
);

CREATE TABLE ProductSupplier (
    ProductID CHAR(5),
    SupplierID CHAR(3),
    UnitPrice DECIMAL(10,2),
    QuantityInStock INT,
    PRIMARY KEY (ProductID, SupplierID),
    FOREIGN KEY (ProductID) REFERENCES Product(ProductID),
    FOREIGN KEY (SupplierID) REFERENCES Supplier(SupplierID)
);
```

2.8 Observations and Notes

- Composite PK is necessary because the same product may be supplied by multiple suppliers.
- Insert, update, and delete anomalies are removed.
- Referential integrity ensures valid references between ProductSupplier, Product, Supplier, and Category.
- All tables are in 3NF.

3 Question 3: Key Concepts and Weak Entities

3.1 1. Distinction among Primary Key, Candidate Key, and Superkey

- **Superkey:** A set of attributes that uniquely identifies a tuple. Can be redundant. **Example:** $\{StudentID, Email\}$ is a superkey of **Student**.
- **Candidate Key:** Minimal superkey; no attribute can be removed without losing uniqueness. **Example:** $\{StudentID\}$ alone is a candidate key for **Student**.
- **Primary Key:** One of the candidate keys chosen to uniquely identify tuples. **Example:** $\{StudentID\}$ as primary key.

3.2 2. Examples of Weak Entities

- **Membership** - Reason: Cannot be uniquely identified by its own attributes alone; needs **StudentID + SocietyID** as identifying relationship.
- **EventParticipation** (if tracking which students attend which events) - Reason: Needs EventID + StudentID as identifying relationship; cannot exist independently of the event and student.

3.3 3. Cardinality Notation

- The relationship where one entity is associated with exactly one related entity, and the related entity can be associated with at most one entity is called a 1:1 relationship. - **(Min, Max) notation:** - For Entity A to Entity B: (1,1) - For Entity B to Entity A: (0,1) or (1,1) depending on participation.

—

4 Question 4: University Society Management System

4.1 Entities and Attributes

- **Campus** Attributes: CampusID (PK), CampusName, Location Description: Each campus hosts multiple societies.
- **Society** Attributes: SocietyID (PK), SocietyName, Type, FoundedYear, CampusID (FK) Description: Each society is associated with exactly one campus.
- **Student** Attributes: StudentID (PK), FirstName, LastName, Email, DateOfBirth, Major, YearOfStudy Description: Students can join multiple societies and hold roles.
- **Role** Attributes: RoleID (PK), RoleName Description: Each role can be assigned to multiple students within societies.

- **Event** Attributes: EventID (PK), EventName, Date, Location, Description, SocietyID (FK) Description: Each event is organized by exactly one society.

4.2 Relationships and Cardinality

- **Campus** – **Society**: One-to-Many (1:N) - Each campus hosts multiple societies. - Society must belong to exactly one campus.

- **Student** – **Society**: Many-to-Many (M:N) - Students can join multiple societies. - Societies can have multiple students. - Associative entity required: **Membership(StudentID, SocietyID, RoleID)**

- **Role** – **Student**: Many-to-Many via Membership - Each student can have multiple roles in different societies. - Each role can be assigned to multiple students.

- **Society** – **Event**: One-to-Many (1:N) - Each society organizes multiple events. - Each event is organized by one society.

4.3 ERD Description (Structural Constraints)

- **Campus** 1:N **Society** - Mandatory for society (every society must belong to a campus). - Optional for campus (a campus may exist without any societies).

- **Student** M:N **Society** through **Membership** - Membership table contains {StudentID, SocietyID, RoleID} as composite PK. - RoleID is a FK referencing **Role** entity.

- **Society** 1:N **Event** - Event table contains SocietyID as FK. - Composite PK could be {EventID}.

Summary of Keys:

- Primary keys (PK) identified for each entity.

- Foreign keys (FK) establish relationships.

- Composite PKs used for associative entities (Membership) to handle M:N relationships.

4.4 ERD Diagram (Optional TikZ Representation)

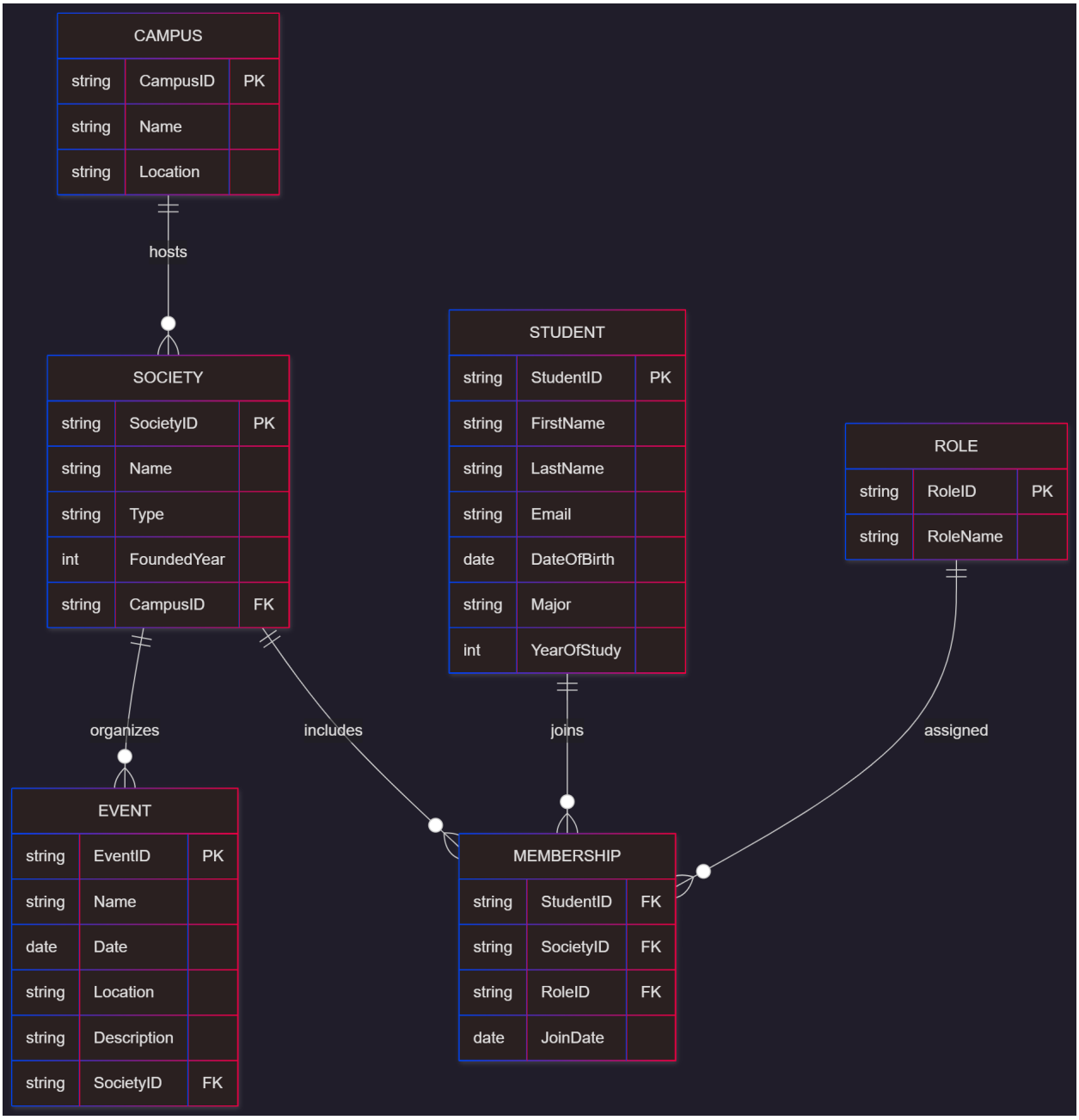

Figure 1: Entity-Relationship Diagram (Version 1)

5. Problem statement and assumptions

The input is a single unnormalized table containing movie-related information. The visible attributes (columns) are:

MovieID, MovieTitle, GenreID, Genre, Language, DateRelease, Re-

leaseCountry, MovieRating, ActorID, ActorName, ReviewerID, ReviewerName, NumberOfRatings, ActorRole

Important assumptions:

1. A **movie** can have many actors. An **actor** can appear in many movies.
2. A **reviewer** may review many actors or movies.
3. The column "NumberOfRatings" is the total number of ratings for a movie (depends only on MovieID).
4. "ActorRole" is the role that a particular actor plays in a particular movie (depends on MovieID and ActorID).
5. GenreID → Genre.
6. MovieRating depends on the reviewer: {MovieID, ReviewerID} → MovieRating.
7. Each displayed row corresponds to a particular (MovieID, ActorID, ReviewerID) combination.

Primary key for the unnormalized table

No single attribute uniquely identifies every row:

- MovieID repeats (a movie has multiple actors / reviews).
- ActorID repeats (an actor can appear in many movies).
- ReviewerID repeats (a reviewer reviews many items).

The smallest safe composite key that uniquely identifies a row is:

$$\mathbf{PK}_{UNF} = (\text{MovieID},\ \text{ActorID},\ \text{ReviewerID})$$

Functional dependencies (FDs)

1. MovieID → {MovieTitle, GenreID, Language, DateRelease, ReleaseCountry, NumberOfRatings}
2. GenreID → Genre
3. ActorID → ActorName
4. {MovieID, ActorID} → ActorRole
5. ReviewerID → ReviewerName
6. {MovieID, ReviewerID} → MovieRating
7. {MovieID, ActorID, ReviewerID} → (no extra attributes beyond above)

Anomalies in the unnormalized table

Insertion anomaly:

- Cannot insert a new actor role for a movie unless we repeat movie details.
- Cannot insert a new movie rating without a reviewer row.

Update anomaly:

- If ActorName is misspelled, it must be updated in all rows.
- Updating MovieRating for a reviewer requires multiple rows to remain consistent.

Deletion anomaly:

- Deleting the last actor or reviewer row for a movie may remove movie-level data like NumberOfRatings or MovieTitle.

Normalization: Step-by-step to BCNF

1NF

All attributes are atomic (no nested structures), so 1NF is satisfied.

2NF

Primary key: (MovieID, ActorID, ReviewerID) Check for partial dependencies:

- MovieTitle, GenreID, Language, DateRelease, ReleaseCountry, NumberOfRatings depend only on MovieID $\rightarrow$ partial dependency.
- ActorName depends only on ActorID $\rightarrow$ partial dependency.
- ReviewerName depends only on ReviewerID $\rightarrow$ partial dependency.
- ActorRole depends on (MovieID, ActorID) $\rightarrow$ full dependency.
- MovieRating depends on (MovieID, ReviewerID) $\rightarrow$ full dependency.

Decomposition to remove partial dependencies:

- Movie(MovieID, MovieTitle, GenreID, Language, DateRelease, ReleaseCountry, NumberOfRatings)
- Genre(GenreID, Genre)
- Actor(ActorID, ActorName)
- Reviewer(ReviewerID, ReviewerName)
- ActorInMovie(MovieID, ActorID, ActorRole)
- MovieRating(MovieID, ReviewerID, MovieRating)
- Review(MovieID, ActorID, ReviewerID)

3NF

Check transitive dependencies:

- Movie: GenreID → Genre stored separately → no transitive dependency.
- All other tables: no transitive dependencies (each non-key attribute depends on the whole key or a single attribute) → 3NF satisfied.

BCNF

Check each relation:

- Movie: MovieID is key → BCNF.
- Genre: GenreID is key → BCNF.
- Actor: ActorID is key → BCNF.
- Reviewer: ReviewerID is key → BCNF.
- ActorInMovie: Key = (MovieID, ActorID), ActorRole depends on both → BCNF.
- MovieRating: Key = (MovieID, ReviewerID), MovieRating depends on both → BCNF.
- Review: Key = (MovieID, ActorID, ReviewerID), no extra FDs → BCNF.

Final BCNF Schema (relations, PKs, FKs)

- **Genre(GenreID PK, Genre)**
- **Movie(MovieID PK, MovieTitle, GenreID FK → Genre.GenreID, Language, DateRelease, ReleaseCountry, NumberOfRatings)**
- **Actor(ActorID PK, ActorName)**
- **Reviewer(ReviewerID PK, ReviewerName)**
- **ActorInMovie(MovieID PK_part, ActorID PK_part, ActorRole)**
  PK = (MovieID, ActorID); FK MovieID → Movie(MovieID); FK ActorID → Actor(ActorID)
- **MovieRating(MovieID PK_part, ReviewerID PK_part, MovieRating)**
  PK = (MovieID, ReviewerID); FK MovieID → Movie(MovieID); FK ReviewerID → Reviewer(ReviewerID)
- **Review(MovieID PK_part, ActorID PK_part, ReviewerID PK_part)**
  PK = (MovieID, ActorID, ReviewerID); FK MovieID → Movie(MovieID); FK ActorID → Actor(ActorID); FK ReviewerID → Reviewer(ReviewerID)

SQL DDL for the BCNF schema

```
CREATE TABLE Genre (
  GenreID VARCHAR(10) PRIMARY KEY,
  Genre VARCHAR(50) NOT NULL
);

CREATE TABLE Movie (
  MovieID VARCHAR(10) PRIMARY KEY,
  MovieTitle VARCHAR(200),
  GenreID VARCHAR(10),
  Language VARCHAR(50),
  DateRelease DATE,
  ReleaseCountry VARCHAR(50),
  NumberOfRatings INT,
  FOREIGN KEY (GenreID) REFERENCES Genre(GenreID)
);

CREATE TABLE Actor (
  ActorID VARCHAR(10) PRIMARY KEY,
  ActorName VARCHAR(200)
);

CREATE TABLE Reviewer (
  ReviewerID VARCHAR(10) PRIMARY KEY,
  ReviewerName VARCHAR(200)
);

CREATE TABLE ActorInMovie (
  MovieID VARCHAR(10),
  ActorID VARCHAR(10),
  ActorRole VARCHAR(50),
  PRIMARY KEY (MovieID, ActorID),
  FOREIGN KEY (MovieID) REFERENCES Movie(MovieID),
  FOREIGN KEY (ActorID) REFERENCES Actor(ActorID)
);

CREATE TABLE MovieRating (
  MovieID VARCHAR(10),
  ReviewerID VARCHAR(10),
  MovieRating INT,
  PRIMARY KEY (MovieID, ReviewerID),
  FOREIGN KEY (MovieID) REFERENCES Movie(MovieID),
  FOREIGN KEY (ReviewerID) REFERENCES Reviewer(ReviewerID)
);
```

```
CREATE TABLE Review (
  MovieID VARCHAR(10),
  ActorID VARCHAR(10),
  ReviewerID VARCHAR(10),
  PRIMARY KEY (MovieID, ActorID, ReviewerID),
  FOREIGN KEY (MovieID) REFERENCES Movie(MovieID),
  FOREIGN KEY (ActorID) REFERENCES Actor(ActorID),
  FOREIGN KEY (ReviewerID) REFERENCES Reviewer(ReviewerID)
);
```

Summary / Notes

- Functional dependencies were updated to reflect that MovieRating depends on (MovieID, ReviewerID) and NumberOfRatings depends on MovieID.
- The BCNF decomposition resolves partial and transitive dependencies.
- This schema preserves all original data and avoids insertion, update, and deletion anomalies.

6. Relational Algebra to SQL Conversion

Relational Algebra is a formal language used to query relational databases. Each operation has a specific meaning, which can be translated into SQL. Below we demonstrate this process with an example.

6.1 Example Query

Relational Algebra:

$$\pi_{FNAME,LNAME,SALARY}(\sigma_{DNO=5}(EMPLOYEE))$$

Meaning: This query selects all employees who work in department number 5 and returns only their first name, last name, and salary. - $\sigma_{DNO=5}(EMPLOYEE)$: Selection operation, chooses rows where 'DNO = 5'. - $\pi_{FNAME,LNAME,SALARY}(\ldots)$: Projection operation, selects only the columns 'FNAME', 'LNAME', and 'SALARY'.

Equivalent SQL Query:

```
SELECT FNAME, LNAME, SALARY
FROM EMPLOYEE
WHERE DNO = 5;
```

6.2 Another Example

Relational Algebra:

$$\sigma_{SALARY>3000}(\pi_{FNAME,LNAME,SALARY}(EMPLOYEE))$$

Meaning: Select the first name, last name, and salary of employees whose salary is greater than 3000.

SQL Query:

```
SELECT FNAME, LNAME, SALARY
FROM EMPLOYEE
WHERE SALARY > 3000;
```

6.3 Example Query 3: Using UNION

Problem: Retrieve the social security numbers (SSN) of all employees who either work in department 5 or directly supervise an employee who works in department 5.

Relational Algebra Steps:

$$\text{DEP5\_EMPS} \leftarrow \sigma_{DNO=5}(\text{EMPLOYEE})$$
$$\text{RESULT1} \leftarrow \pi_{SSN}(\text{DEP5\_EMPS})$$
$$\text{RESULT2(SSN)} \leftarrow \pi_{SUPERSSN}(\text{DEP5\_EMPS})$$
$$\text{RESULT} \leftarrow \text{RESULT1} \cup \text{RESULT2}$$

Explanation:

- $\sigma_{DNO=5}$(EMPLOYEE): Select all employees who work in department 5.
- π_{SSN}(DEP5_EMPS): Project their SSNs $\rightarrow$ RESULT1.
- $\pi_{SUPERSSN}$(DEP5_EMPS): Project the SSNs of their supervisors $\rightarrow$ RESULT2.
- RESULT1 $\cup$ RESULT2: Union operation returns all SSNs that are in either RESULT1, RESULT2, or both.

Equivalent SQL Query:

```
SELECT SSN
FROM EMPLOYEE
WHERE DNO = 5
UNION
SELECT SUPERSSN
FROM EMPLOYEE
WHERE DNO = 5;
```

7. Relational Algebra: Cartesian Product Example

The **Cartesian Product** ($\times$) operation combines every tuple of one relation with every tuple of another relation. While it is rarely meaningful by itself, it becomes useful when combined with **selection** (σ) to filter relevant combinations.

7.1 Example: Female Employees and Their Dependents

Step 1: Select Female Employees

$$\text{FEMALE\_EMPS} \leftarrow \sigma_{SEX='F'}(\text{EMPLOYEE})$$

Meaning: Selects all employees whose sex is female.

SQL:

```
CREATE VIEW FEMALE_EMPS AS
SELECT *
FROM EMPLOYEE
WHERE SEX = 'F';
```

Step 2: Project Employee Names and SSN

$$\text{EMPNAMES} \leftarrow \pi_{FNAME,LNAME,SSN}(\text{FEMALE\_EMPS})$$

Meaning: Keeps only the first name, last name, and SSN of female employees.

SQL:

```
CREATE VIEW EMPNAMES AS
SELECT FNAME, LNAME, SSN
FROM FEMALE_EMPS;
```

Step 3: Cartesian Product with DEPENDENT Table

$$\text{EMP\_DEPENDENTS} \leftarrow \text{EMPNAMES} \times \text{DEPENDENT}$$

Meaning: Combines every employee with every dependent. This by itself is not meaningful because it includes unrelated pairs.

Step 4: Select Actual Employee-Dependent Pairs

$$\text{ACTUAL\_DEPS} \leftarrow \sigma_{SSN=ESSN}(\text{EMP\_DEPENDENTS})$$

Meaning: Keeps only the tuples where the employee's SSN matches the dependent's ESSN.

SQL:

```
CREATE VIEW ACTUAL_DEPS AS
SELECT E.FNAME, E.LNAME, D.DEPENDENT_NAME
FROM EMPNAMES E, DEPENDENT D
WHERE E.SSN = D.ESSN;
```

Step 5: Project Desired Columns

$$\text{RESULT} \leftarrow \pi_{FNAME,LNAME,DEPENDENT\_NAME}(\text{ACTUAL\_DEPS})$$

Meaning: Returns only the first name, last name, and dependent name of female employees with dependents.

SQL:

```
SELECT FNAME, LNAME, DEPENDENT_NAME
FROM ACTUAL_DEPS;
```

8. Binary Relational Operations: Natural Join

The **Natural Join** ($*$) is a binary relational operation that combines two relations based on attributes with the same name. It automatically matches tuples where the values of these common attributes are equal and eliminates duplicate columns in the result.

8.1 Example 1: Joining Department with Department Locations

Relational Algebra:

$$\text{DEPT\_LOCS} \leftarrow \text{DEPARTMENT} * \text{DEPT\_LOCATIONS}$$

Meaning: - The only common attribute is `DNUMBER`. - Implicit join condition: `DEPARTMENT.DNUMBER = DEPT_LOCATIONS.DNUMBER` - The result contains all attributes from both relations, but `DNUMBER` appears only once.

SQL Equivalent:

```
SELECT *
FROM DEPARTMENT NATURAL JOIN DEPT_LOCATIONS;
```

8.2 Example 2: Natural Join on Multiple Attributes

Relational Algebra:

$$Q \leftarrow R(A, B, C, D) * S(C, D, E)$$

Meaning: - Common attributes: `C` and `D`. - Implicit join condition: `R.C = S.C AND R.D = S.D` - The result keeps only one copy of each common attribute:

$$Q(A, B, C, D, E)$$

SQL Equivalent:

```
SELECT R.A, R.B, R.C, R.D, S.E
FROM R
NATURAL JOIN S;
```

Note: Natural join is convenient because you do not need to explicitly specify the join condition; it automatically uses all attributes with the same name in both relations.

9. Binary Relational Operations: Division

The **Division** operation is used to find tuples in one relation that are associated with *all* tuples in another relation.

9.1 Definition

Let $R(Z)$ and $S(X)$ be two relations, where $X \subset Z$ and $Y = Z - X$. The division $R \div S$ returns a relation $T(Y)$ containing all tuples t in Y such that:

$$r[Y] = t \quad \text{and} \quad r[X] = s$$

9.2 Example

Relations:

$R(Student, Course)$

Student	Course
Alice	Math
Alice	CS
Bob	Math
Bob	CS
Charlie	Math

$S(Course)$

Course
Math
CS

We want $R \div S$, i.e., find students enrolled in all courses in S.

9.3 Solution

- Alice: Courses = {Math, CS} → includes all courses in S → included - Bob: Courses = {Math, CS} → includes all courses in S → included - Charlie: Courses = {Math} → missing CS → not included

Result:

Student
Alice
Bob

9.4 Equivalent SQL Queries

Method 1: Using NOT EXISTS

```
SELECT R.Student
FROM R
WHERE NOT EXISTS (
    SELECT S.Course
```

```
    FROM S
    WHERE S.Course NOT IN (
        SELECT R2.Course
        FROM R AS R2
        WHERE R2.Student = R.Student
    )
);
```

Method 2: Using GROUP BY and HAVING

```
SELECT Student
FROM R
GROUP BY Student
HAVING COUNT(DISTINCT Course) = (SELECT COUNT(*) FROM S);
```

Explanation: - First method: Select students for whom no course in S is missing in R. - Second method: Count the number of courses each student has and compare with total courses in S. - Both queries return the same result: Alice, Bob

10. Summary of Relational Algebra Operations

Operation	**Symbol**	**Meaning**	**RA Example**
Selection	σ	Selects rows satisfying a condition	$\sigma_{DNO=5}$(EMPLOYEE)
Projection	π	Selects specific columns	$\pi_{FNAME,LNAME,SALARY}$ (EMPLOYEE)
Renaming	ρ	Renames relation or attributes	$\rho_{EMP(FNAME,LNAME,SALARY,DNO)}$ (EMPLOYEE)
Assignment	$\leftarrow$	Stores RA expression in a temporary relation	$FEMALE\_EMPS \leftarrow \sigma_{SEX='F'}$(EMPLOYEE)
Union	$\cup$	Tuples in R or S (no duplicates)	$R \cup S$
Intersection	$\cap$	Tuples common to R and S	$R \cap S$
Difference	$-$	Tuples in R but not in S	$R - S$
Cartesian Product	$\times$	Combine every tuple from R with every tuple from S	$R \times S$
Theta Join	$\bowtie$	Combine tuples satisfying condition	$R \bowtie_{R.B=S.D} S$
Equi Join	$\bowtie$	Join using equality condition	$R \bowtie_{A=B} S$
Natural Join	$*$	Join on all attributes with same name, remove duplicates	$DEPARTMENT * DEPT\_LOCATIONS$
Division	$\div$	Tuples in R related to all tuples in S	$R(Student, Course) \div S(Course)$

Table 3: Summary of Relational Algebra Operations

Q11: Relational Algebra and SQL Queries

Marks: 10 Estimated Time: 30 minutes

11. University Database Schema

Table	Attributes	Primary Key / Foreign Keys
Student	(StudentId, StudentName)	PK: StudentId
Faculty	(FacultyId, ProfName)	PK: FacultyId
Course	(CourseId, CourseName)	PK: CourseId
Qualified	(FacultyId, CourseId, DateQualified)	PK: (FacultyId, CourseId), FK: FacultyId → Faculty, CourseId → Course
Section	(SectionNo, Semester, CourseId)	PK: SectionNo, FK: CourseId → Course
Registration	(SectionID, StudentNo, Semester)	PK: (SectionID, StudentNo), FK: SectionID → Section, StudentNo → Student

—

(a) Display all courses for which Professor Miley has been qualified.

Relational Algebra

$$\pi_{C.CourseName}(\sigma_{F.ProfName='Miley'}((\rho_F(Faculty) \bowtie_{F.FacultyId=Q.FacultyId} \rho_Q(Qualified)) \bowtie_{Q.CourseId=C.CourseId} \rho_C(Course)))$$

SQL Query

```sql
SELECT C.CourseName
FROM Faculty F
JOIN Qualified Q ON F.FacultyId = Q.FacultyId
JOIN Course C ON Q.CourseId = C.CourseId
WHERE F.ProfName = 'Miley';
```

—

(b) Which instructors are qualified to teach CT220?

Relational Algebra

$$\pi_{F.ProfName}(\sigma_{C.CourseId='CT220'}((\rho_F(Faculty) \bowtie_{F.FacultyId=Q.FacultyId} \rho_Q(Qualified)) \bowtie_{Q.CourseId=C.CourseId} \rho_C(Course)))$$

SQL Query

```sql
SELECT F.ProfName
FROM Faculty F
JOIN Qualified Q ON F.FacultyId = Q.FacultyId
JOIN Course C ON Q.CourseId = C.CourseId
WHERE C.CourseId = 'CT220';
```

—

(c) Is any instructor qualified to teach CT220 and not qualified to teach CS208?

Relational Algebra

$$(\pi_{F.ProfName}(\sigma_{C.CourseId='CT220'}((\rho_F(Faculty) \bowtie_{F.FacultyId=Q.FacultyId} \rho_Q(Qualified)) \bowtie_{Q.CourseId=C.CourseId} \rho_C(Course))))$$
$$-$$
$$(\pi_{F.ProfName}(\sigma_{C.CourseId='CS208'}((\rho_F(Faculty) \bowtie_{F.FacultyId=Q.FacultyId} \rho_Q(Qualified)) \bowtie_{Q.CourseId=C.CourseId} \rho_C(Course))))$$

SQL Query

```sql
SELECT F.ProfName
FROM Faculty F
JOIN Qualified Q ON F.FacultyId = Q.FacultyId
JOIN Course C ON Q.CourseId = C.CourseId
WHERE C.CourseId = 'CT220'
AND F.ProfName NOT IN (
    SELECT F2.ProfName
    FROM Faculty F2
```

```
    JOIN Qualified Q2 ON F2.FacultyId = Q2.FacultyId
    JOIN Course C2 ON Q2.CourseId = C2.CourseId
    WHERE C2.CourseId = 'CS208'
);
```

—

(d) How many students are enrolled in section 1345 during semester I-2012?

Relational Algebra

$$\gamma_{COUNT(StudentNo)}(\sigma_{R.SectionID=1345 \wedge R.Semester='I\text{-}2012'}(\rho_R(Registration)))$$

SQL Query

```
SELECT COUNT(R.StudentNo) AS TotalStudents
FROM Registration R
WHERE R.SectionID = 1345
AND R.Semester = 'I-2012';
```

—

(e) How many students are enrolled in CT220 during semester I-2012?

Relational Algebra

$$\gamma_{COUNT(StudentNo)}(\sigma_{C.CourseId='CT220' \wedge R.Semester='I\text{-}2012'}((\rho_C(Course) \bowtie_{C.CourseId=S.CourseId} \rho_S(Section)) \bowtie_{S.SectionNo=R.SectionID} \rho_R(Registration)))$$

SQL Query

```
SELECT COUNT(R.StudentNo) AS TotalStudents
FROM Course C
JOIN Section S ON C.CourseId = S.CourseId
JOIN Registration R ON S.SectionNo = R.SectionID
WHERE C.CourseId = 'CT220'
AND R.Semester = 'I-2012';
```

—